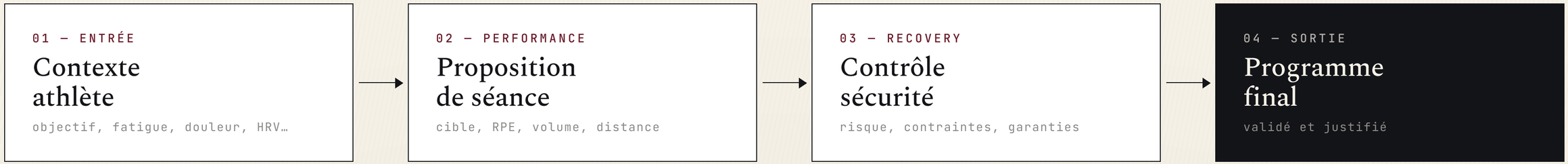
PRÉSENTATION DU PROJET

TrainMind

Recommandation sportive argumentée par système multi-agent.



ENTRÉE

objectif · sommeil · fatigue
douleur · HRV · ACWR · historique

PROBLÈME

Progresser sans ignorer *le risque du jour.*

IDÉE

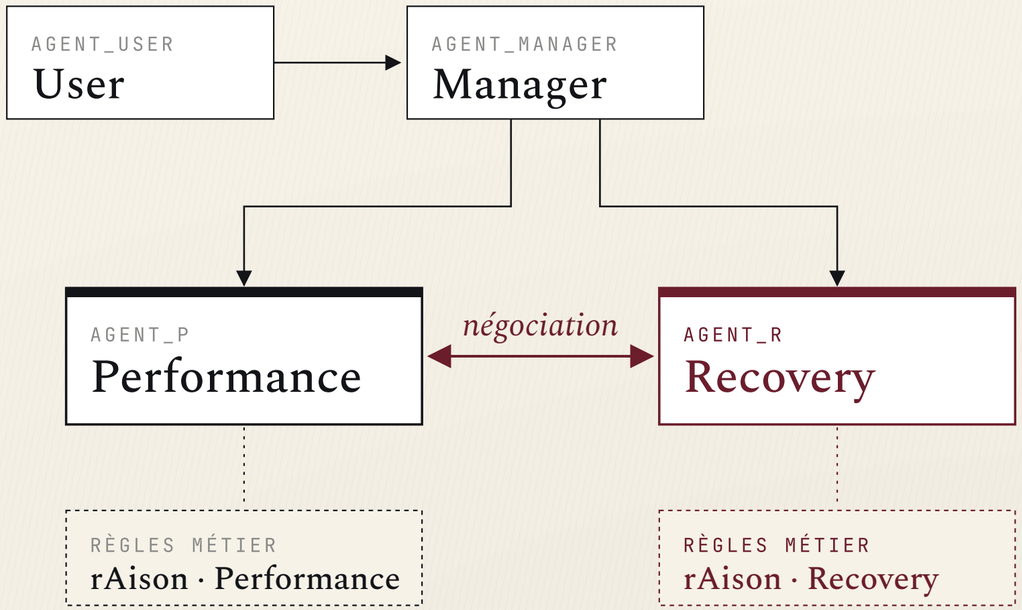
Le programme final vient d'une *négociation* entre performance et récupération.

« Une séance utile n'est acceptable que si elle reste sûre aujourd'hui. »

ARCHITECTURE DES AGENTS

Quatre agents, *deux expertises.*

User et Manager orchestrent. Performance et Recovery négocient.



`Agent_User`	Saisit le *contexte* et affiche le résultat.
`Agent_Manager`	Coordonne la *session* de négociation.
`Agent_P`	Propose un plan de *performance* — séance utile pour progresser.
`Agent_R`	Analyse le *risque* et demande des garanties.

« *Performance défend la progression ; Recovery défend la sécurité.* »

PILE TECHNIQUE · NÉGOCIATION

JADE orchestre, *rAison décide.*

La plateforme route les messages ; la logique métier reste hors du code Java.

PILE TECHNIQUE

JADE — PLATEFORME

Agents, **behaviours**, conversation et automate de négociation.

rAison · *Performance* — RÈGLES MÉTIER

Séance, cible, **RPE**, volume, distance.

rAison · *Recovery* — RÈGLES MÉTIER

Risque, contraintes, validation.

FIPA *ACL* — PROTOCOLE

`propose` · `reject-proposal` · `query-if` · `inform` · `accept-proposal`

MINI-SÉQUENCE DE NÉGOCIATION

	AGENT_P · PERFORMANCE	AGENT_R · RECOVERY
i.	**`propose`** Plan : course, `8×400 m`, RPE 8. →	
ii.	←	**`query-if · reject-proposal`** Demande une *contrainte* — règle d'arrêt si douleur.
iii.	**`propose`** Plan corrigé : `6×400 m`, RPE 7, stop si douleur. →	
iv.	←	**`accept-proposal`** Validation — `plan_validated`.

DÉMO → *Observer comment le plan évolue réellement entre les tours.*

« Je passe à la démonstration pour montrer une proposition qui évolue réellement entre les tours. »